



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Práctica 4:

Filtro pasa bajas, filtro pasa altas, filtro pasa bandas-
Pico a Matlab.

Integrantes:

Espinoza Matamoros Percival Ulises - 320025561

Flores Colin Victor Jaziel - 320266083

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Microcomputadoras

Grupo: 03 - Semestre: 2026-2

Calf: _____



1. Objetivo:

Diseñar e implementar un simulador de funciones de transferencia discretas de primer y segundo orden mediante una Raspberry Pi Pico 2W programada en MicroPython, capaz de aplicar filtros IIR (pasa bajas, pasa altas y pasa banda) en tiempo real a una señal de entrada muestreada a 8 kHz, comunicándose con MATLAB a través de un enlace Bluetooth HC-05 para la recepción de coeficientes y el envío de datos de entrada/salida para su visualización y análisis espectral.

2. Desarrollo:

Parámetros iniciales:

- Frecuencia de muestreo: $F_s = 8000$ Hz
- Periodo de muestreo: $T = \frac{1}{F_s} = \frac{1}{8000} = 125 \mu s$
- Frecuencia de corte pasa bajas: $f_c = 85$ Hz
- Frecuencia de corte pasa altas: $f_c = 480$ Hz
- Frecuencia central (pasa banda): $f_o = 580$ Hz
- Ancho de banda (pasa banda): $BW = 45$ Hz
- Número de muestras por captura: 500
- Tipo de filtro: Butterworth IIR

2.1. Diseño del Filtro Pasa Bajas de Primer Orden

El prototipo analógico Butterworth de primer orden tiene la función de transferencia:

$$H(s) = \frac{\omega_c}{s + \omega_c} \quad (1)$$

Se calcula la frecuencia de corte pre-distorsionada para compensar el efecto de la transformación bilineal:



$$\Omega_c = 2F_s \tan\left(\frac{\pi f_c}{F_s}\right) = 2(8000) \tan\left(\frac{\pi \cdot 85}{8000}\right) = 16000 \tan(0.03337) \quad (2)$$

$$\Omega_c = 16000 \times 0.033386 = 534.18 \text{ rad/s} \quad (3)$$

Aplicación de la Transformación Bilineal

Sustituyendo $s = 2F_s \cdot \frac{z-1}{z+1}$ en $H(s)$ con ω_c reemplazado por Ω_c :

$$H(z) = \frac{\Omega_c}{2F_s \cdot \frac{z-1}{z+1} + \Omega_c} \quad (4)$$

Multiplicando numerador y denominador por $(z+1)$:

$$H(z) = \frac{\Omega_c(z+1)}{2F_s(z-1) + \Omega_c(z+1)} \quad (5)$$

Expandiendo el denominador:

$$H(z) = \frac{\Omega_c(z+1)}{(2F_s + \Omega_c)z + (\Omega_c - 2F_s)} \quad (6)$$

Dividiendo todo entre $(2F_s + \Omega_c)$:

$$H(z) = \frac{\frac{\Omega_c}{2F_s + \Omega_c}(z+1)}{z - \frac{2F_s - \Omega_c}{2F_s + \Omega_c}} \quad (7)$$

Cálculo de los Coeficientes

$$A_0 = A_1 = \frac{\Omega_c}{2F_s + \Omega_c} = \frac{534.18}{16000 + 534.18} = \frac{534.18}{16534.18} \approx 0.03230 \quad (8)$$

$$A_2 = 0 \quad (9)$$

$$B_1 = \frac{2F_s - \Omega_c}{2F_s + \Omega_c} = \frac{16000 - 534.18}{16534.18} = \frac{15465.82}{16534.18} \approx 0.93540 \quad (10)$$

$$B_2 = 0 \quad (11)$$

Ganancia en DC

En DC ($z = 1$):

$$H(1) = \frac{A_0 + A_1}{1 - B_1} = \frac{0.03230 + 0.03230}{1 - 0.93540} = \frac{0.06460}{0.06460} = 1.0 \checkmark \quad (12)$$

Esto confirma que el filtro tiene ganancia unitaria en DC, como se espera de un pasa bajas.

2.1.1. Función de Transferencia:

$$H(z) = \frac{0.03230 z + 0.03230}{z - 0.93540} = \frac{0.03230(1 + z^{-1})}{1 - 0.93540 z^{-1}} \quad (13)$$

Diagrama de Bloques

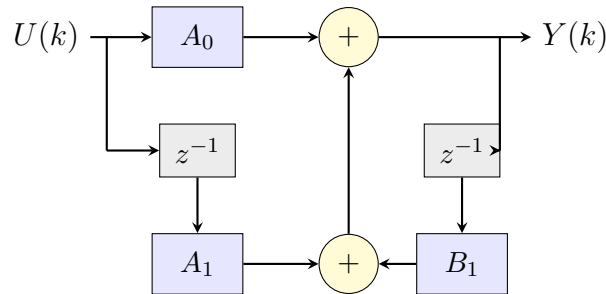


Figura 1: Diagrama de bloques del filtro pasa bajas de primer orden (Forma Directa I).

2.1.2. Frecuencias de entradas en matlab:

Tabla 1: Frecuencias de prueba para el filtro pasa bajas ($f_c = 85$ Hz)

Frecuencia	Relación con f_c	Comportamiento esperado
20 Hz	$f \ll f_c$	Paso sin atenuación
50 Hz	$f < f_c$	Paso con mínima atenuación, curva exponencial visible
85 Hz	$f = f_c$	Atenuación de -3 dB (70.7 %)
200 Hz	$f > f_c$	Atenuación significativa
500 Hz	$f \gg f_c$	Atenuación fuerte



2.2. Diseño del Filtro Pasa Altas de Primer Orden

El prototipo analógico Butterworth pasa altas de primer orden:

$$H(s) = \frac{s}{s + \omega_c} \quad (14)$$

Para el filtro pasa altas se selecciona $f_c = 480$ Hz. La frecuencia pre-distorsionada es:

$$\Omega_c = 2F_s \tan\left(\frac{\pi \cdot 480}{8000}\right) = 16000 \tan(0.18850) = 16000 \times 0.19089 = 3054.24 \text{ rad/s} \quad (15)$$

Sustituyendo en la transformación bilineal:

$$H(z) = \frac{2F_s \cdot \frac{z-1}{z+1}}{2F_s \cdot \frac{z-1}{z+1} + \Omega_c} \quad (16)$$

Multiplicando por $(z + 1)$:

$$H(z) = \frac{2F_s(z - 1)}{2F_s(z - 1) + \Omega_c(z + 1)} = \frac{2F_s(z - 1)}{(2F_s + \Omega_c)z + (\Omega_c - 2F_s)} \quad (17)$$

Dividiendo entre $(2F_s + \Omega_c)$:

$$H(z) = \frac{\frac{2F_s}{2F_s + \Omega_c}(z - 1)}{z - \frac{2F_s - \Omega_c}{2F_s + \Omega_c}} \quad (18)$$

Cálculo de los Coeficientes

$$A_0 = \frac{2F_s}{2F_s + \Omega_c} = \frac{16000}{19054.24} \approx 0.83970 \quad (19)$$

$$A_1 = -A_0 = -0.83970 \quad (20)$$

$$A_2 = 0 \quad (21)$$

$$B_1 = \frac{2F_s - \Omega_c}{2F_s + \Omega_c} = 0.67940 \quad (22)$$

$$B_2 = 0 \quad (23)$$

Ganancia en DC y Nyquist

En DC ($z = 1$):

$$H(1) = \frac{A_0 + A_1}{1 - B_1} = \frac{0.83970 - 0.83970}{1 - 0.67940} = \frac{0}{0.32060} = 0 \checkmark \quad (24)$$

En Nyquist ($z = -1$):

$$H(-1) = \frac{A_0(-1) + A_1}{-1 - (-B_1)} = \frac{-0.83970 - 0.83970}{-1 - 0.67940} = \frac{-1.67940}{-1.67940} = 1.0 \checkmark \quad (25)$$

Función de Transferencia Final

$$H(z) = \frac{0.83970z - 0.83970}{z - 0.67940} = \frac{0.83970(1 - z^{-1})}{1 - 0.67940z^{-1}} \quad (26)$$

Diagrama de Bloques

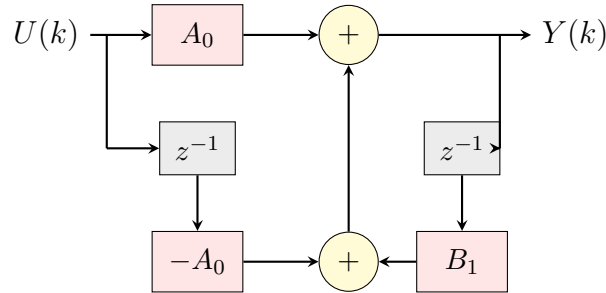


Figura 2: Diagrama de bloques del filtro pasa altas de primer orden (Forma Directa I).

Frecuencias de entradas en matlab:

Tabla 2: Frecuencias de prueba para el filtro pasa altas ($f_c = 480$ Hz)

Frecuencia	Relación con f_c	Comportamiento esperado
50 Hz	$f \ll f_c$	Atenuación fuerte
200 Hz	$f < f_c$	Atenuación significativa
480 Hz	$f = f_c$	Atenuación de -3 dB (70.7 %)
1000 Hz	$f > f_c$	Paso con mínima atenuación
2000 Hz	$f \gg f_c$	Paso sin atenuación



2.3. Filtro Pasa Banda de Segundo Orden

Parámetros

- Frecuencia central: $f_o = 580$ Hz
- Ancho de banda: $BW = 45$ Hz
- Frecuencias de corte: $f_{c1} = f_o - BW/2 = 557.5$ Hz, $f_{c2} = f_o + BW/2 = 602.5$ Hz

Se aplica pre-warping a ambas frecuencias de corte:

$$\Omega_1 = 2F_s \tan\left(\frac{\pi f_{c1}}{F_s}\right) = 16000 \tan\left(\frac{\pi \cdot 557.5}{8000}\right) = 16000 \tan(0.21893) \quad (27)$$

$$= 16000 \times 0.22237 = 3557.9 \text{ rad/s} \quad (28)$$

$$\Omega_2 = 2F_s \tan\left(\frac{\pi f_{c2}}{F_s}\right) = 16000 \tan\left(\frac{\pi \cdot 602.5}{8000}\right) = 16000 \tan(0.23658) \quad (29)$$

$$= 16000 \times 0.24203 = 3872.5 \text{ rad/s} \quad (30)$$

$$\Omega_0 = \sqrt{\Omega_1 \cdot \Omega_2} = \sqrt{3557.9 \times 3872.5} = \sqrt{13\,779\,000} \approx 3712.0 \text{ rad/s} \quad (31)$$

$$\Omega_0^2 \approx 13\,778\,944 \quad (32)$$

$$BW_w = \Omega_2 - \Omega_1 = 3872.5 - 3557.9 = 314.6 \text{ rad/s} \quad (33)$$

Función de Transferencia del Pasa Banda

El prototipo analógico Butterworth pasa banda de segundo orden:

$$H(s) = \frac{BW_w \cdot s}{s^2 + BW_w \cdot s + \Omega_0^2} \quad (34)$$

Aplicando la transformación bilineal $s = K \frac{z-1}{z+1}$ con $K = 2F_s = 16000$:

$$H(z) = \frac{BW_w \cdot K \cdot \frac{z-1}{z+1}}{\left(K \frac{z-1}{z+1}\right)^2 + BW_w \cdot K \cdot \frac{z-1}{z+1} + \Omega_0^2} \quad (35)$$



Multiplicando numerador y denominador por $(z + 1)^2$:

$$H(z) = \frac{BW_w \cdot K (z^2 - 1)}{K^2(z - 1)^2 + BW_w \cdot K (z^2 - 1) + \Omega_0^2(z + 1)^2} \quad (36)$$

2.3.1. Expansión del Denominador

$$K^2(z - 1)^2 = K^2 z^2 - 2K^2 z + K^2 \quad (37)$$

$$BW_w \cdot K (z^2 - 1) = BW_w \cdot K z^2 - BW_w \cdot K \quad (38)$$

$$\Omega_0^2(z + 1)^2 = \Omega_0^2 z^2 + 2\Omega_0^2 z + \Omega_0^2 \quad (39)$$

Sumando término a término:

$$D(z) = \underbrace{(K^2 + BW_w K + \Omega_0^2)}_{\alpha} z^2 + \underbrace{(-2K^2 + 2\Omega_0^2)}_{\beta} z + \underbrace{(K^2 - BW_w K + \Omega_0^2)}_{\gamma} \quad (40)$$

Sustituyendo valores numéricos ($K = 16000$, $K^2 = 256\,000\,000$):

$$\alpha = 256\,000\,000 + 5\,033\,600 + 13\,778\,944 = 274\,812\,544 \quad (41)$$

$$\beta = -512\,000\,000 + 27\,557\,888 = -484\,442\,112 \quad (42)$$

$$\gamma = 256\,000\,000 - 5\,033\,600 + 13\,778\,944 = 264\,745\,344 \quad (43)$$

Coefficientes del Filtro Discreto

Dividiendo numerador y denominador entre α :



$$A_0 = \frac{BW_w \cdot K}{\alpha} = \frac{314.6 \times 16000}{274\,812\,544} = \frac{5\,033\,600}{274\,812\,544} \approx 0.01832 \quad (44)$$

$$A_1 = 0 \quad (45)$$

$$A_2 = -A_0 = -0.01832 \quad (46)$$

$$B_1 = -\frac{\beta}{\alpha} = -\frac{-484\,442\,112}{274\,812\,544} = \frac{484\,442\,112}{274\,812\,544} \approx 1.76283 \quad (47)$$

$$B_2 = -\frac{\gamma}{\alpha} = -\frac{264\,745\,344}{274\,812\,544} \approx -0.96337 \quad (48)$$

Verificación

En DC ($z = 1$):

$$H(1) = \frac{A_0(1) + 0 + A_2(1)}{1 - B_1 - B_2} = \frac{0.01832 - 0.01832}{1 - 1.76283 + 0.96337} = \frac{0}{0.20054} = 0 \quad \checkmark \quad (49)$$

En Nyquist ($z = -1$):

$$H(-1) = \frac{A_0 + A_2}{1 + B_1 - B_2} = \frac{0}{1 + 1.76283 + 0.96337} = 0 \quad \checkmark \quad (50)$$

El filtro pasa banda rechaza correctamente tanto las señales DC como las de frecuencia Nyquist.

2.3.2. Función de Transferencia Final

$$H(z) = \frac{0.01832 z^2 - 0.01832}{z^2 - 1.76283 z + 0.96337} = \frac{0.01832(1 - z^{-2})}{1 - 1.76283 z^{-1} + 0.96337 z^{-2}} \quad (51)$$

Diagrama de Bloques

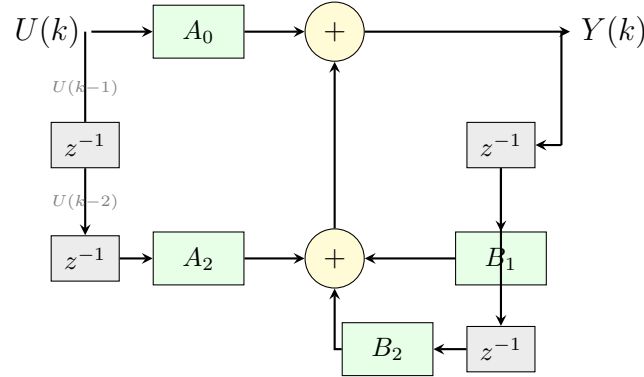


Figura 3: Diagrama de bloques del filtro pasa banda de segundo orden (Forma Directa I).
Nota: $A_1 = 0$, por lo que no hay rama para $U(k-1)$.

2.4. Coeficientes de todos los filtros;

Tabla 3: Coeficientes de los filtros IIR diseñados ($F_s = 8000$ Hz)

Coeficiente	Pasa Bajas	Pasa Altas	Pasa Banda
f_c o f_o	85 Hz	480 Hz	580 Hz
BW	—	—	45 Hz
A_0	0.03230	0.83970	0.01832
A_1	0.03230	-0.83970	0.0
A_2	0.0	0.0	-0.01832
B_1	0.93540	0.67940	1.76283
B_2	0.0	0.0	-0.96337

- **Pasa Bajas** ($f_c = 85$ Hz): Primer orden, ganancia unitaria en DC, atenuación de -3 dB en f_c . Con f_c baja, la señal de salida presenta una curva exponencial visible al filtrar ondas cuadradas. Probarlo con 50 Hz (paso con forma exponencial), 85 Hz (-3 dB) y 500 Hz (atenuación fuerte).
- **Pasa Altas** ($f_c = 480$ Hz): Primer orden, rechazo total en DC, ganancia unitaria en Nyquist. Elimina componentes de baja frecuencia, dejando solo transitorios (flancos) de la onda cuadrada. Probarlo con 50 Hz (atenuación), 480 Hz (-3 dB) y 2000 Hz (paso).

- **Pasa Banda** ($f_o = 580$ Hz, $BW = 45$ Hz): Segundo orden, selectivo en frecuencia. Extrae la componente a 580 Hz de la señal, produciendo una salida sinusoidal. Probarlo con 580 Hz (paso), 50 Hz (atenuación) y 2000 Hz (atenuación).

2.5. Diagrama de Conexión del Circuito

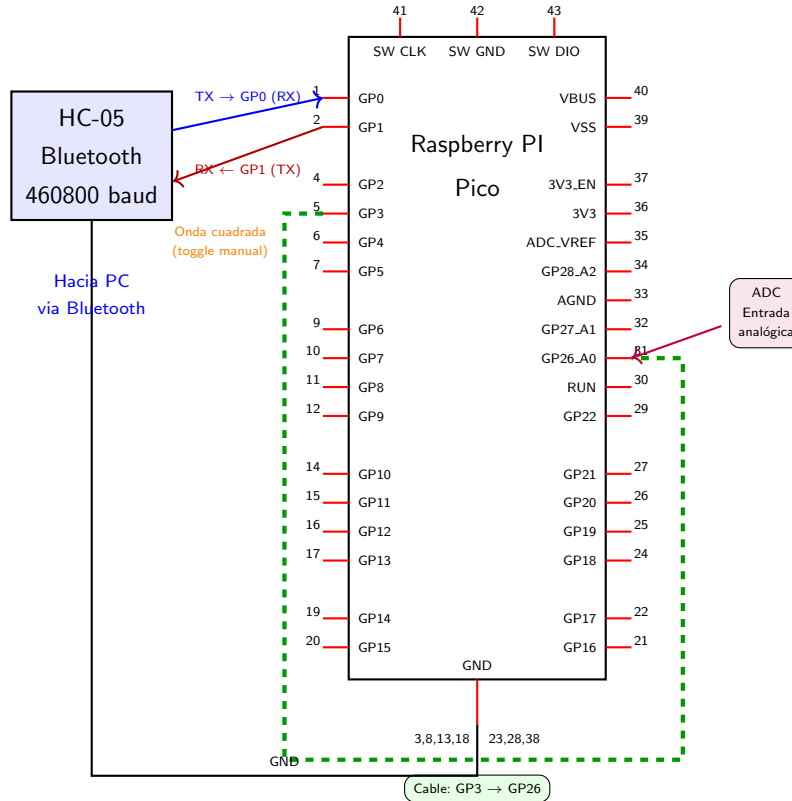


Figura 4: Diagrama de conexión del circuito: Raspberry Pi Pico 2W con módulo HC-05 y lazo de retroalimentación.

Funcionamiento del circuito:

1. MATLAB envía los coeficientes del filtro (A_0, A_1, A_2, B_1, B_2) y la frecuencia de la señal de prueba a través de Bluetooth.
2. La Pico configura la frecuencia de la onda cuadrada generada por toggle manual en el hilo del Core 2.
3. Al recibir el comando de captura ($0x52$), el Core 1 ejecuta la ecuación en diferencias a



8 kHz durante 500 muestras.

4. La onda cuadrada generada en el Pin 3 se conecta externamente al Pin 26 (ADC), sirviendo como señal de entrada.
5. Los vectores de entrada $U(k)$ y salida $Y(k)$ se envían de vuelta a MATLAB para su visualización.

2.6. Protocolo de Comunicación UART

La comunicación entre MATLAB y la Raspberry Pi Pico se realiza mediante UART a 460800 baudios a través del módulo HC-05. Se definen dos tipos de tramas:

MATLAB envía una trama de 25 bytes con la siguiente estructura:

Tabla 4: Estructura de la trama de coeficientes

Byte(s)	Campo	Formato	Descripción
0	Encabezado	0x55 (“U”)	Identificador de trama
1–4	A_0	float32 LE	Coeficiente de $U(k)$
5–8	A_1	float32 LE	Coeficiente de $U(k-1)$
9–12	A_2	float32 LE	Coeficiente de $U(k-2)$
13–16	B_1	float32 LE	Coeficiente de $Y(k-1)$
17–20	B_2	float32 LE	Coeficiente de $Y(k-2)$
21–24	FREQ	float32 LE	Frecuencia de prueba (Hz)

MATLAB envía el byte 0x52 para iniciar la captura de datos.

2.7. Explicación del Código:

Listing 1: Código MicroPython del simulador de filtros IIR

```
1 import machine
2 machine.freq(300_000_000) # Overclock a 300 MHz
3
4 from machine import Pin, ADC, UART
5 from time import sleep_ms, sleep_us, ticks_us, ticks_diff, ticks_add
6 import _thread
```



```
7 import struct
8
9 ADC0 = ADC(Pin(26))
10 Pin3 = Pin(3, Pin.OUT)
11 UART0 = UART(0, baudrate=460800, tx=Pin(0), rx=Pin(1))
12
13 conversion_factor = 3.3 / 65535
14 MUESTRAS = 1000
15 ENTRADAS = [0]*MUESTRAS
16 SALIDAS = [0]*MUESTRAS
17 En_Bytes = bytearray(4000)
18
19 CTE_A0, CTE_A1, CTE_A2 = 0.0303, 0.0303, 0.0
20 CTE_B1, CTE_B2 = 0.9394, 0.0
21 FREQ = 50
22 PERIODO = 1/FREQ
23 HALF = int((PERIODO*1000000)/2)
24
25 def Onda_Cuadrada():
26     while True:
27         Pin3.on()
28         sleep_us(HALF)
29         Pin3.off()
30         sleep_us(HALF)
31
32 @micropython.native
33 def capturar(a0, a1, a2, b1, b2):
34     uk = uk1 = uk2 = 0.0
35     yk = yk1 = yk2 = 0.0
36     adc = ADC0.read_u16
37     cf = conversion_factor
38     ent = ENTRADAS
39     sal = SALIDAS
40     _tu = ticks_us
41     _td = ticks_diff
42     _ta = ticks_add
```



```
43     _su = sleep_us
44
45     t_next = _tu()
46     for i in range(500):
47         yk = a0*uk + a1*uk1 + a2*uk2 + b1*yk1 + b2*yk2
48         uk2 = uk1
49         uk1 = uk
50         yk2 = yk1
51         yk1 = yk
52         uk = adc() * cf
53         ent[i] = uk
54         sal[i] = yk
55         t_next = _ta(t_next, 125)
56         dt = _td(t_next, _tu())
57         if dt > 0:
58             _su(dt)
59
60 _thread.start_new_thread(Onda_Cuadrada, ())
61
62 while True:
63     if UART0.any() > 0:
64         sleep_ms(100)
65         LLAVE = UART0.read()
66         if LLAVE and LLAVE[0] == 0x55:
67             # Decodificar 6 flotantes IEEE 754 LE
68             CTE_A0 = struct.unpack('<f', LLAVE[1:5])[0]
69             CTE_A1 = struct.unpack('<f', LLAVE[5:9])[0]
70             CTE_A2 = struct.unpack('<f', LLAVE[9:13])[0]
71             CTE_B1 = struct.unpack('<f', LLAVE[13:17])[0]
72             CTE_B2 = struct.unpack('<f', LLAVE[17:21])[0]
73             FREQ = struct.unpack('<f', LLAVE[21:25])[0]
74             HALF = int((1/FREQ * 1000000) / 2)
75         if LLAVE and 0x52 in LLAVE:
76             capturar(CTE_A0, CTE_A1, CTE_A2, CTE_B1, CTE_B2)
77             for i in range(500):
78                 struct.pack_into('f', En_Bytes, i*8, ENTRADAS[i])
```



```
79         struct.pack_into('f', En_Bytes, i*8+4, SALIDAS[i])
80     for j in range(0, 4000, 50):
81         UART0.write(En_Bytes[j:j+50])
82         sleep_ms(60)
```

2.7.1. Configuración del Hardware

Listing 2: Configuración de la frecuencia del reloj

```
1 import machine
2 machine.freq(300_000_000) # Overclock a 300 MHz
```

Se incrementa la frecuencia del procesador de 125 MHz (valor por defecto) a 300 MHz para mejorar la velocidad y precisión de los cálculos y del control de tiempo en la función `capturar()` a 8 kHz.

Listing 3: Limpieza de PWM residual

```
1 # Limpiar cualquier PWM residual en Pin 3
2 try:
3     from machine import PWM
4     _p = PWM(Pin(3))
5     _p.deinit()
6 except:
7     pass
```

Limpia cualquier señal PWM que pudiera haber quedado configurada en el Pin 3 de ejecuciones anteriores. Esto evita conflictos porque el mismo pin se usará como salida digital GPIO.

Listing 4: Inicialización de periféricos y comunicación

```
1 ADC0 = ADC(Pin(26)) # Entrada analógica en GP26
2 Pin3 = Pin(3, Pin.OUT) # Salida digital para la onda cuadrada
3 UART0 = UART(0, baudrate=460800, tx=Pin(0), rx=Pin(1)) # Comunicación Bluetooth
```

- ADC0: Convierte la señal analógica de entrada (proveniente del Pin 3 conectado externamente al Pin 26).



- Pin3: Genera la onda cuadrada que sirve como señal de prueba.
- UART0: Canal de comunicación serie con el módulo HC-05 a 460,800 baudios.

2.7.2. Variables Globales

Listing 5: Definición de factores de conversión y buffers

```
1 conversion_factor = 3.3 / 65535      # Convierte lecturas ADC (0-65535)
   a voltios (0-3.3 V)
2 MUESTRAS = 1000
3 ENTRADAS = [0] * MUESTRAS           # Buffer para almacenar U(k)
4 SALIDAS  = [0] * MUESTRAS           # Buffer para almacenar Y(k)
5 En_Bytes = bytearray(4000)          # Espacio en memoria para enviar
   los datos por USB
```

Los buffers ENTRADAS y SALIDAS guardan temporalmente los datos obtenidos durante la captura antes de enviarlos. Por otro lado, En_Bytes reserva un espacio fijo en memoria para transmitir los datos por USB, evitando retrasos o interrupciones durante la ejecución del sistema en tiempo real.

Listing 6: Coeficientes del filtro y temporización de la onda cuadrada

```
1 # Coeficientes iniciales (pasa bajas, fc \approx 85 Hz)
2 CTE_A0, CTE_A1, CTE_A2 = 0.0303, 0.0303, 0.0
3 CTE_B1, CTE_B2 = 0.9394, 0.0
4 FREQ = 50                      # Frecuencia inicial de la onda cuadrada (Hz)
5 PERIODO = 1/FREQ
6 HALF = int((PERIODO * 1_000_000) / 2) # Semiperíodo en
   microsegundos
```

- CTE_A0, CTE_A1, CTE_A2, CTE_B1, CTE_B2: Coeficientes iniciales para la implementación de un filtro digital paso bajo con una frecuencia de corte (f_c) de aproximadamente 85 Hz.
- FREQ: Define la frecuencia inicial de la onda cuadrada, establecida por defecto en 50 Hz.
- PERIODO y HALF: Determinan matemáticamente el período de la señal y el semiperíodo



(HALF) convertido a microsegundos, necesario para controlar con precisión los tiempos de conmutación del pin digital.

2.7.3. Función Onda_Cuadrada() - Núcleo 2 (hilo secundario)

Listing 7: Generación de señal en el hilo secundario

```
1 def Onda_Cuadrada():
2     while True:
3         Pin3.on()
4         sleep_us(HALF)
5         Pin3.off()
6         sleep_us(HALF)
```

Genera una onda cuadrada con un ciclo de trabajo del 50 % en el Pin3. Esta función se ejecuta en el Núcleo 2 (*Core 2*) de la Raspberry Pi Pico 2W usando la biblioteca `_thread`, por lo que trabaja de manera independiente al programa principal.

El valor de HALF, que define el semiperíodo de la señal, se actualiza automáticamente cuando MATLAB envía una nueva frecuencia. Como HALF es una variable global que se revisa continuamente, la frecuencia de la onda puede cambiarse en tiempo real sin detener la ejecución del hilo secundario.

2.7.4. Función capturar() - Motor del Filtro Digital

Listing 8: Declaración y decorador de la función principal

```
1 @micropython.native
2 def capturar(a0, a1, a2, b1, b2):
```

La instrucción `@micropython.native` hace que la función se ejecute como código máquina nativo en lugar de código interpretado, aumentando considerablemente su velocidad de ejecución.

Listing 9: Inicialización de variables de estado

```
1     # Inicialización de estados del filtro (condiciones iniciales =
      0)
2     uk = uk1 = uk2 = 0.0      # Muestras de entrada actuales y
      anteriores
```



```
3 yk = yk1 = yk2 = 0.0      # Muestras de salida actuales y
    anteriores
```

Listing 10: Optimización mediante referencias locales

```
1 # Referencias locales para máximo rendimiento (evitar búsqueda
    de atributos globales)
2 adc = ADC0.read_u16
3 cf = conversion_factor
4 ent = ENTRADAS
5 sal = SALIDAS
6 _tu = ticks_us
7 _td = ticks_diff
8 _ta = ticks_add
9 _su = sleep_us
```

Este método de guardar referencias en variables locales es una optimización importante en MicroPython, ya que acceder a variables locales es aproximadamente 4× más rápido que acceder a variables globales o atributos de objetos.

Listing 11: Bucle principal de procesamiento del filtro

```
1 t_next = _tu()          # Marca de tiempo del inicio
2 for i in range(500):
3     # 1. APLICAR LA ECUACIÓN EN DIFERENCIAS
4     yk = a0*uk + a1*uk1 + a2*uk2 + b1*yk1 + b2*yk2
5
6     # 2. ACTUALIZAR ESTADOS (desplazamiento de registros de
        retardo)
7     uk2 = uk1
8     uk1 = uk
9     yk2 = yk1
10    yk1 = yk
11
12    # 3. LEER NUEVA MUESTRA DEL ADC
13    uk = adc() * cf      # Convierte a voltios
14
15    # 4. GUARDAR EN BUFFERS
```



```
16     ent[i] = uk
17     sal[i] = yk
18
19     # 5. CONTROL DE TEMPORIZACIÓN (periodo exacto de 125
        microseconds = 8 kHz)
20     t_next = _ta(t_next, 125)
21     dt = _td(t_next, _tu())
22     if dt > 0:
23         _su(dt)                # Esperar el tiempo restante del
        periodo
```

La ecuación en diferencias implementada es la forma general:

$$Y(k) = A_0 \cdot U(k) + A_1 \cdot U(k-1) + A_2 \cdot U(k-2) + B_1 \cdot Y(k-1) + B_2 \cdot Y(k-2)$$

El control del tiempo es muy importante: en lugar de usar un `sleep_us(125)` fijo (lo que podría generar errores por el tiempo que tarda el programa en ejecutarse), el sistema calcula cuánto tiempo falta para la siguiente muestra y solo espera ese intervalo restante. De esta manera, se mantiene un periodo de muestreo estable de 125 μ s.

2.7.5. Bucle Principal - Protocolo de Comunicación

Listing 12: Inicialización del hilo secundario y bucle principal

```
1 _thread.start_new_thread(Onda_Cuadrada, ())    # Lanzar generador de
        onda en Core 2
2
3 while True:
4     if UART0.any() > 0:
5         sleep_ms(100)                # Espera a que se reciban todos los
        datos
6         LLAVE = UART0.read()
```



Recepción de Coeficientes (datos 0x55)

Listing 13: Decodificación de los datos de coeficientes

```
1  if LLAVE and LLAVE[0] == 0x55:
2      # Decodificar 6 floats IEEE 754 little-endian (4 bytes
      # cada uno)
3      CTE_A0 = struct.unpack('<f', LLAVE[1:5])[0]
4      CTE_A1 = struct.unpack('<f', LLAVE[5:9])[0]
5      CTE_A2 = struct.unpack('<f', LLAVE[9:13])[0]
6      CTE_B1 = struct.unpack('<f', LLAVE[13:17])[0]
7      CTE_B2 = struct.unpack('<f', LLAVE[17:21])[0]
8      FREQ   = struct.unpack('<f', LLAVE[21:25])[0]
9      HALF   = int((1/FREQ * 1_000_000) / 2)      # Actualizar
      semiperiodo
```

Los 25 bytes recibidos tienen la siguiente estructura:

Byte(s)	Dato	Descripción
0	0x55	Código de inicio del mensaje
1–4	A0	Valor decimal de tipo float32
5–8	A1	Valor decimal de tipo float32
9–12	A2	Valor decimal de tipo float32
13–16	B1	Valor decimal de tipo float32
17–20	B2	Valor decimal de tipo float32
21–24	FREQ	Frecuencia en formato float32

Tabla 5: Distribución de los datos recibidos y su significado.

Inicio de la Captura de Datos (0x52)

Listing 14: Ejecución de captura y envío de datos

```
1  if LLAVE and 0x52 in LLAVE:
2      # Realizar la captura usando los coeficientes actuales
3      capturar(CTE_A0, CTE_A1, CTE_A2, CTE_B1, CTE_B2)
4
5      # Guardar U(k) y Y(k) en formato float32
```



```
6         for i in range(500):
7             struct.pack_into('f', En_Bytes, i*8, ENTRADAS[i])
8             struct.pack_into('f', En_Bytes, i*8+4, SALIDAS[i])
9
10            # Enviar los datos en bloques de 50 bytes
11            for j in range(0, 4000, 50):
12                UART0.write(En_Bytes[j:j+50])
13                sleep_ms(60)      # Espera para evitar sobrecargar el
                                   HC-05
```

La transmisión se realiza en bloques de 50 bytes con una pausa de 60 ms entre cada uno. Esto se hace para evitar que el módulo Bluetooth HC-05 se sature y pierda datos al recibir toda la información demasiado rápido.

3. Resultados:

3.1. Filtro Pasa Bajas ($f_c = 85$ Hz)

Para evaluar el funcionamiento del filtro pasa bajas de primer orden, se enviaron desde MATLAB los coeficientes calculados previamente mediante la transformación bilineal ($A_0 = 0.0323$, $A_1 = 0.0323$, $B_1 = 0.9354$). La comunicación Bluetooth con el módulo HC-05 fue exitosa, como se ve en la terminal de comandos, permitiendo actualizar la frecuencia de la onda cuadrada generada en la Raspberry en tiempo real.

```
25 UART0 = UART(0, baudrate=460800, tx=Pin(0), rx=Pin(1))
26
27 conversion_factor = 3.3 / (65535)
28 MUESTRAS = 1000
29 ENTRADAS = [0]*MUESTRAS
30 SALIDAS = [0]*MUESTRAS
31 En_Bytes = bytearray(4000)
32
33 To_CTE_A0 = bytearray(4)
34 To_CTE_A1 = bytearray(4)
35 To_CTE_A2 = bytearray(4)
36 To_CTE_B1 = bytearray(4)
37 To_CTE_B2 = bytearray(4)
38 To_FREQ = bytearray(4)
39
40 CTE_A0 = +0.0303
41 CTE_A1 = +0.0303
```

```
Console x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Nuevos coeficientes cargados
Coeficientes recibidos:
A0 = 0.0323
A1 = 0.0323
A2 = 0.0
B1 = 0.9354
B2 = 0.0
FREQ = 50.0 Hz
Nuevos coeficientes cargados
Coeficientes recibidos:
A0 = 0.0323
A1 = 0.0323
A2 = 0.0
B1 = 0.9354
B2 = 0.0
FREQ = 100.0 Hz
Nuevos coeficientes cargados
Coeficientes recibidos:
A0 = 0.0323
A1 = 0.0323
A2 = 0.0
B1 = 0.9354
B2 = 0.0
FREQ = 100.0 Hz
Nuevos coeficientes cargados
Coeficientes recibidos:
A0 = 0.0323
A1 = 0.0323
A2 = 0.0
B1 = 0.9354
B2 = 0.0
FREQ = 400.0 Hz
```

Figura 5: Consola de MATLAB mostrando el envío de los coeficientes del filtro pasa bajas y las distintas frecuencias de prueba.

A continuación, presentamos las gráficas comparativas entre la señal de entrada $U(k)$ (onda cuadrada azul) y la señal de salida $Y(k)$ (señal filtrada roja) para los diferentes casos de prueba capturados a 50 Hz, 100 Hz y 200 Hz.

3.1.1. Prueba con $f = 50$ Hz ($f < f_c$)

Al aplicar una frecuencia menor a la frecuencia de corte teórica (85 Hz), la señal se encuentra dentro de la banda de paso. En la siguiente observamos que el filtro atenúa los armónicos de alta frecuencia de los flancos de la onda cuadrada, dando como resultado una salida con la clásica curva exponencial de carga y descarga, pero logrando alcanzar casi la amplitud máxima de la señal de entrada.

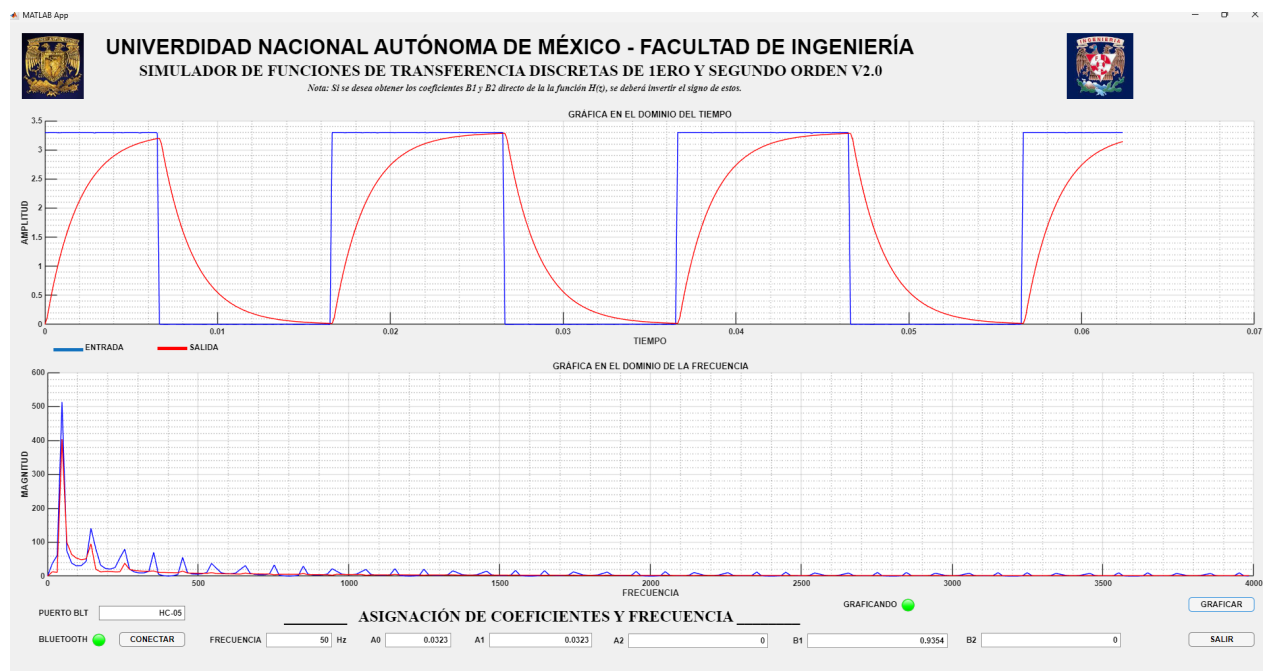


Figura 6: Respuesta del filtro pasa bajas a una entrada de 50 Hz. Se observa el seguimiento de la señal con distorsión exponencial típica de un sistema de primer orden.

3.1.2. Prueba con $f = 100$ Hz ($f > f_c$)

La segunda prueba la hicimos dando una señal de 100 Hz. Al haber superado la frecuencia de corte de 85 Hz, la señal entra en la banda de rechazo del filtro. Y la siguiente imagen demuestra que la señal filtrada roja sufre una atenuación visible y ya no logra alcanzar el

nivel de voltaje de la señal de entrada, debido a que el sistema no tiene tiempo suficiente para cargar antes del siguiente cambio de estado.

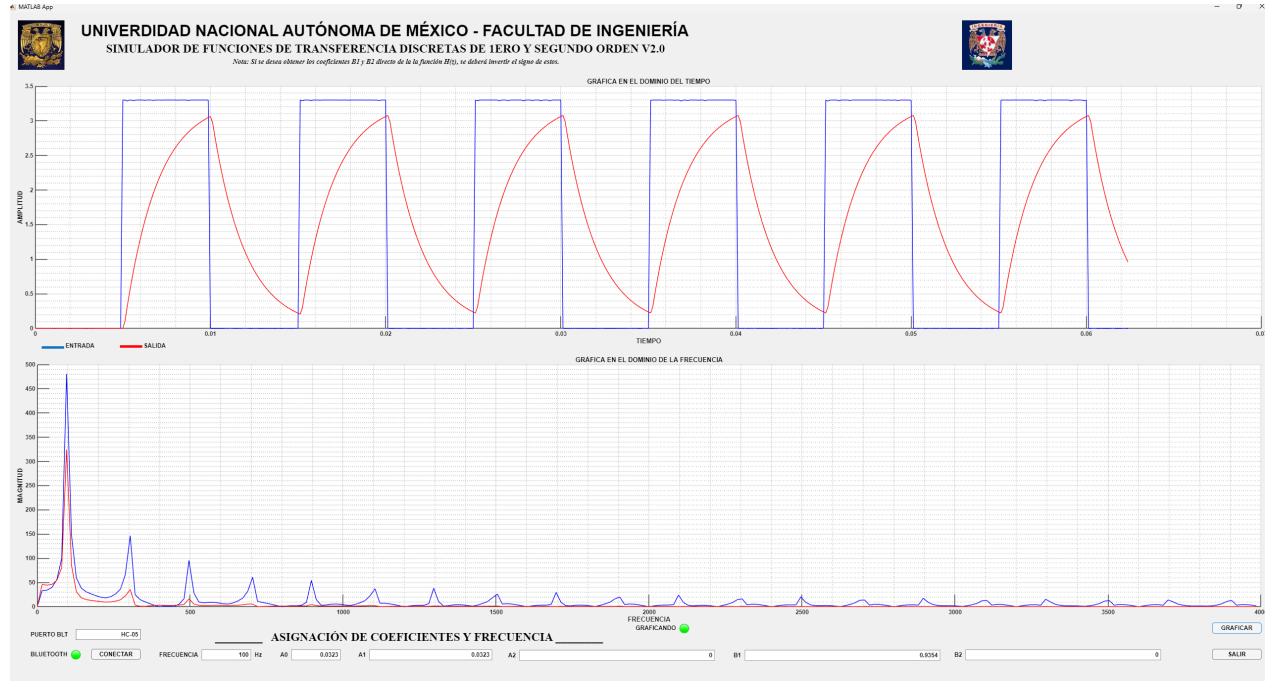


Figura 7: Respuesta del filtro pasa bajas a una entrada de 100 Hz. Se aprecia una atenuación notable al haber superado la frecuencia de corte.

3.1.3. Prueba con $f = 200 \text{ Hz}$ ($f \gg f_c$)

Ahora, se aumentó la frecuencia de la señal de entrada a 200 Hz. Estando considerablemente por encima de la frecuencia de corte, el comportamiento esperado era una atenuación más significativa. En la imagen podemos ver este rechazo, en la salida $Y(k)$ se presenta una amplitud sumamente reducida y se asemeja más como a una onda triangular pequeña, y es aquí donde podemos ver la capacidad del filtro para suprimir componentes de alta frecuencia.

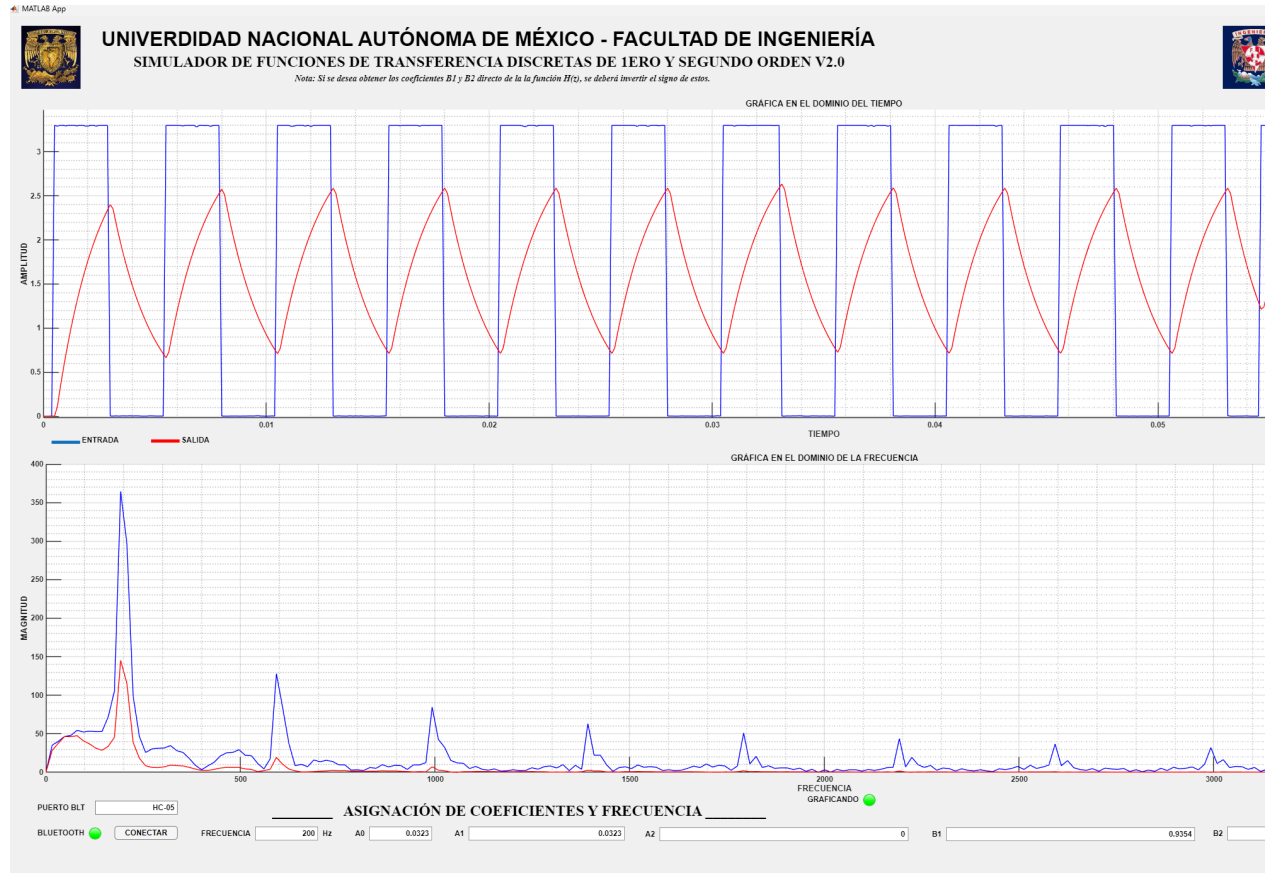


Figura 8: Respuesta del filtro pasa bajas a una entrada de 200 Hz. Muestra una atenuación severa, filtrando casi en su totalidad la señal.

3.2. Filtro Pasa Altas ($f_c = 480$ Hz)

De manera similar al caso anterior, se enviaron desde MATLAB los coeficientes correspondientes al filtro pasa altas de primer orden ($A_0 = 0.8397$, $A_1 = -0.8397$, $B_1 = 0.6794$). En la consola podemos ver la correcta recepción de estos valores y la actualización de las frecuencias a probar.



```
15 try:
16     from machine import PWM
17     _p = PWM(Pin(3))
18     _p.deinit()
19 except:
20     pass
21
22 ADC0 = ADC(Pin(26))
23 Pin3 = Pin(3, Pin.OUT)
24
25 UART0 = UART(0, baudrate=460800, tx=Pin(0), rx=Pin(1))
26
27 conversion_factor = 3.3 / (65535)
28 MUESTRAS = 1000
29 ENTRADAS = [0]*MUESTRAS
30 SALIDAS = [0]*MUESTRAS
31 En_Bytes = bytearray(4000)
32
33 To_CTE_A0 = bytearray(4)
34 To_CTE_A1 = bytearray(4)
35 To_CTE_A2 = bytearray(4)
36 To_CTE_B1 = bytearray(4)
37 To_CTE_B2 = bytearray(4)
38 To_FREQ = bytearray(4)
39
40 CTE_A0 = +0.0303
41 CTE_A1 = +0.0303
```

Console x

```
>>> %Run -c $EDITOR_CONTENT
```

```
MPY: soft reboot
Nuevos coeficientes cargados
Coeficientes recibidos:
A0 = 0.8397
A1 = -0.8397
A2 = 0.0
B1 = 0.6794
B2 = 0.0
FREQ = 50.0 Hz
Nuevos coeficientes cargados
Coeficientes recibidos:
A0 = 0.8397
A1 = -0.8397
A2 = 0.0
B1 = 0.6794
B2 = 0.0
FREQ = 100.0 Hz
Nuevos coeficientes cargados
Coeficientes recibidos:
A0 = 0.8397
A1 = -0.8397
A2 = 0.0
B1 = 0.6794
B2 = 0.0
FREQ = 200.0 Hz
```

Figura 9: Consola de MATLAB mostrando la carga exitosa de los coeficientes del filtro pasa altas y las frecuencias enviadas.

Para este filtro, la frecuencia de corte teórica es de 480 Hz. Dado que las frecuencias de prueba (50 Hz, 100 Hz y 200 Hz) están por debajo de este valor, la componente fundamental de la señal cuadrada se encuentra en la banda de atenuación en los tres casos. Sin embargo, debido a que los flancos abruptos de la onda cuadrada están compuestos por armónicos de muy alta frecuencia, el filtro permitirá el paso de estos transitorios, comportándose como un circuito derivador.

3.2.1. Prueba con $f = 50$ Hz ($f \ll f_c$)

Al aplicar una frecuencia de 50 Hz, estando muy por debajo de la frecuencia de corte, la atenuación de la señal original es muy fuerte. En la imagen se observa claramente que el filtro elimina por completo las partes planas (constantes o de baja frecuencia) de la onda cuadrada. Solo sobreviven los cambios bruscos de estado, generando picos de voltaje que decaen rápidamente a cero entre un flanco y otro.

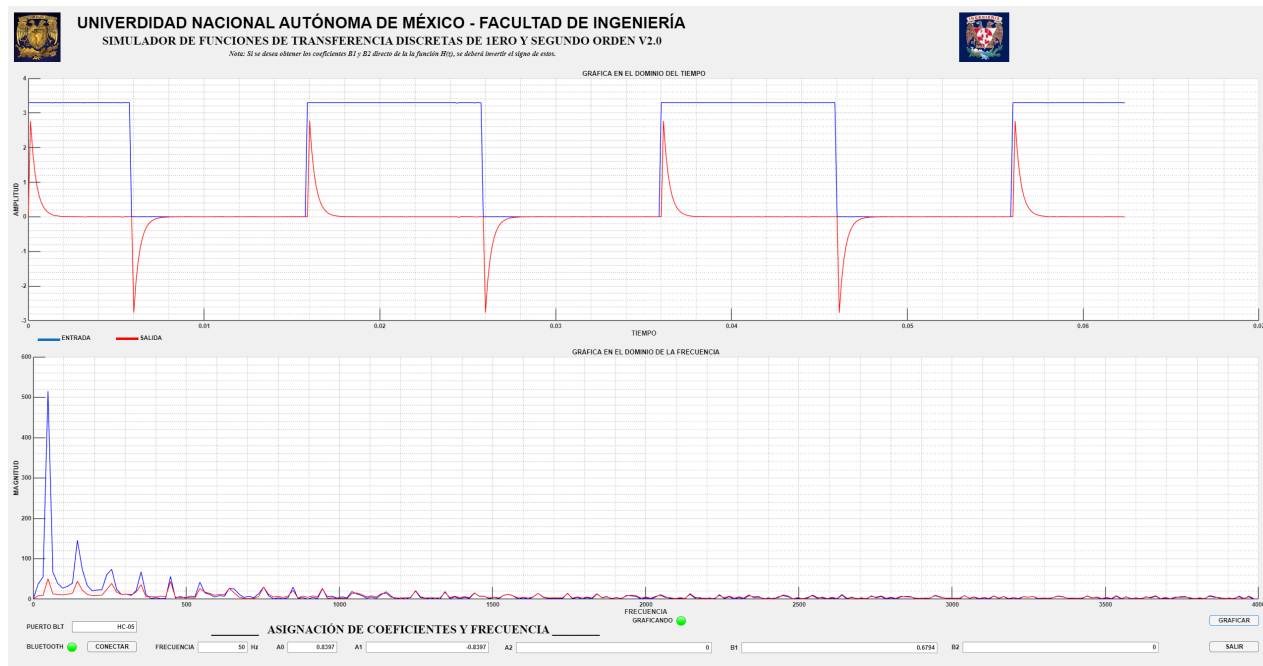


Figura 10: Respuesta del filtro pasa altas a 50 Hz. Se observa una atenuación fuerte de la componente fundamental, dejando pasar únicamente los transitorios de los flancos.

3.2.2. Prueba con $f = 100$ Hz ($f < f_c$)

Al incrementar la frecuencia a 100 Hz, seguimos dentro de la banda de atenuación. El comportamiento es el mismo: la señal filtrada roja muestra exclusivamente las respuestas transitorias en cada cambio de flanco de la señal azul de entrada. Al ser el periodo más corto que en la prueba anterior, los picos se presentan con mayor frecuencia en la gráfica, pero la amplitud de las partes planas sigue siendo rechazada y llevada a cero.

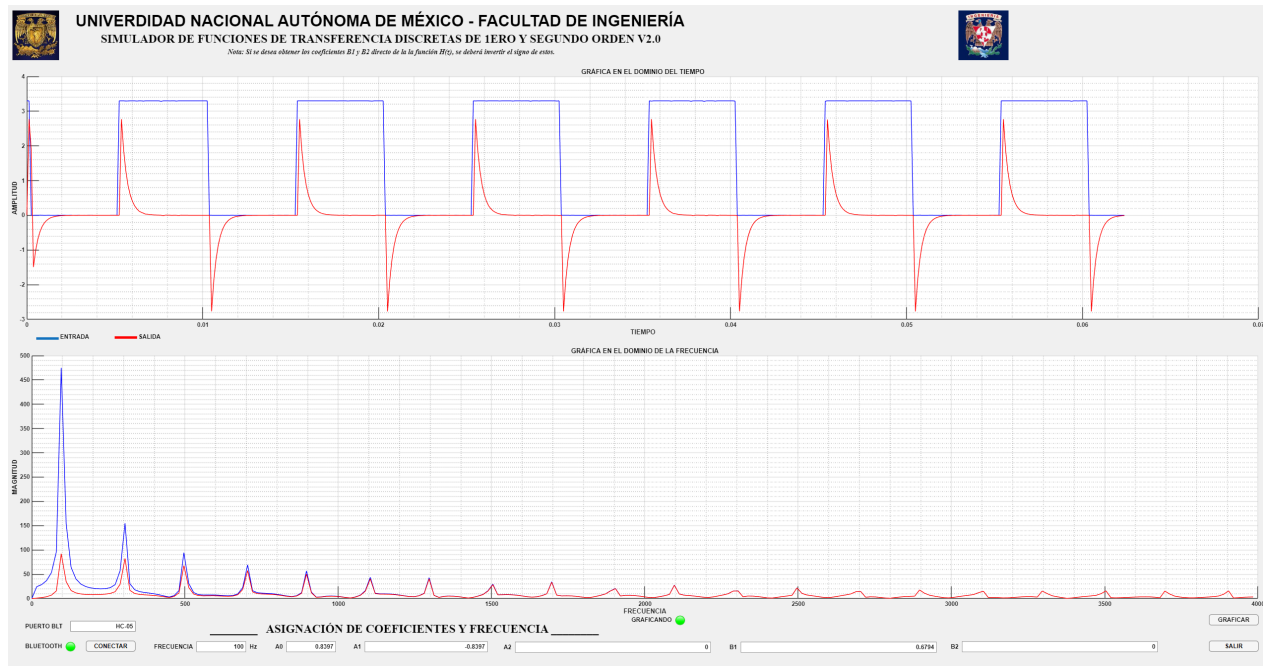


Figura 11: Respuesta del filtro pasa altas a 100 Hz. Las partes planas de la onda cuadrada continúan siendo filtradas exitosamente.

3.2.3. Prueba con $f = 200$ Hz ($f < f_c$)

En la última prueba a 200 Hz, aunque nos acercamos un poco más a la frecuencia de corte, todavía nos encontramos por debajo de los 480 Hz. La gráfica reafirma la estabilidad del filtro: actúa de manera consistente eliminando las componentes de baja frecuencia y permitiendo el paso de los armónicos superiores que conforman las transiciones de la onda cuadrada de entrada.

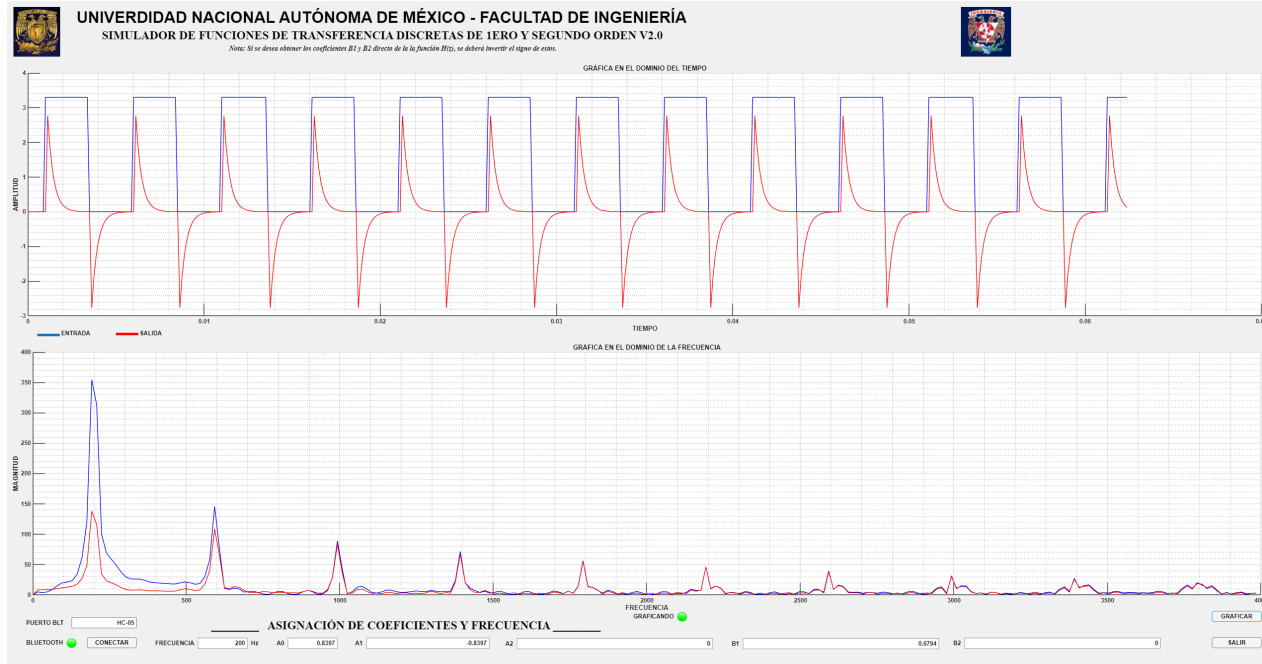


Figura 12: Respuesta del filtro pasa altas a 200 Hz. El comportamiento derivador se mantiene estable, destacando los cambios rápidos de la señal.

3.3. Filtro Pasa Banda ($f_o = 580 \text{ Hz}$, $BW = 45 \text{ Hz}$)

Para la implementación del filtro pasa banda, se enviaron al sistema los coeficientes calculados para obtener una frecuencia central de 580 Hz con un ancho de banda de 45 Hz, estos coeficientes fueron $A_0 = 0.01832$, $A_1 = 0.0$, $A_2 = -0.01832$, $B_1 = 1.76283$, $B_2 = -0.96337$ y de igual forma que en los casos anteriores la comunicación con el módulo HC-05 fue satisfactoria, permitiendo la carga de los parámetros y la sintonización del filtro en tiempo real, según se verifica en la imagen de abajo donde se muestra la consola.



```
FINAL_ECUATION.py x
11 import _thread
12 import struct
13
14 # Limpiar cualquier PWM residual en Pin 3 de ejecuciones anteriores
15 try:
16     from machine import PWM
17     _p = PWM(Pin(3))
18     _p.deinit()
19 except:
20     pass
21
22 ADC0 = ADC(Pin(26))
23 Pin3 = Pin(3, Pin.OUT)
24
25 UART0 = UART(0, baudrate=460800, tx=Pin(0), rx=Pin(1))
26
27 conversion_factor = 3.3 / (65535)
28 MUESTRAS = 1000
29 ENTRADAS = [0]*MUESTRAS
30 SALIDAS = [0]*MUESTRAS
31 En_Bytes = bytearray(4000)
32
33 To_CTE_A0 = bytearray(4)
34 To_CTE_A1 = bytearray(4)
35 To_CTE_A2 = bytearray(4)
36 To_CTE_B1 = bytearray(4)
37 To_CTE_B2 = bytearray(4)
38 To_FREQ = bytearray(4)
39
40 CTE_A0 = +0.0303
41 CTE_A1 = +0.0303
```

```
MPY: soft reboot
Nuevos coeficientes cargados
Coeficientes recibidos:
A0 = 0.01832
A1 = 0.0
A2 = -0.01832
B1 = 1.76283
B2 = -0.96337
FREQ = 50.0 Hz
Nuevos coeficientes cargados
Coeficientes recibidos:
A0 = 0.01832
A1 = 0.0
A2 = -0.01832
B1 = 1.76283
B2 = -0.96337
FREQ = 400.0 Hz
Nuevos coeficientes cargados
Coeficientes recibidos:
A0 = 0.01832
A1 = 0.0
A2 = -0.01832
B1 = 1.76283
B2 = -0.96337
FREQ = 200.0 Hz
Nuevos coeficientes cargados
Coeficientes recibidos:
A0 = 0.01832
A1 = 0.0
A2 = -0.01832
B1 = 1.76283
B2 = -0.96337
FREQ = 100.0 Hz
```

Figura 13: Consola de MATLAB mostrando la carga de los coeficientes para el filtro pasa banda y la recepción de las distintas frecuencias de prueba.

Dado que la frecuencia central de diseño de 580 Hz está fuera del rango de las frecuencias de prueba utilizadas anteriormente, por lo tanto el filtro se comporta de manera selectiva frente a la entrada. A continuación, analizamos el comportamiento del filtro ante diferentes señales.

3.3.1. Prueba con $f = 50$ Hz ($f < f_o$)

Al aplicar una frecuencia de 50 Hz, la cual se encuentra significativamente por debajo de la frecuencia de corte inferior de la banda de paso, observamos que el filtro atenúa drásticamente la señal. Como se aprecia en la figura, la salida $Y(k)$ presenta una amplitud mínima, confirmando que las componentes de baja frecuencia son bloqueadas eficazmente por la sección pasa altas del filtro, dejando pasar únicamente una mínima fracción de la señal original.

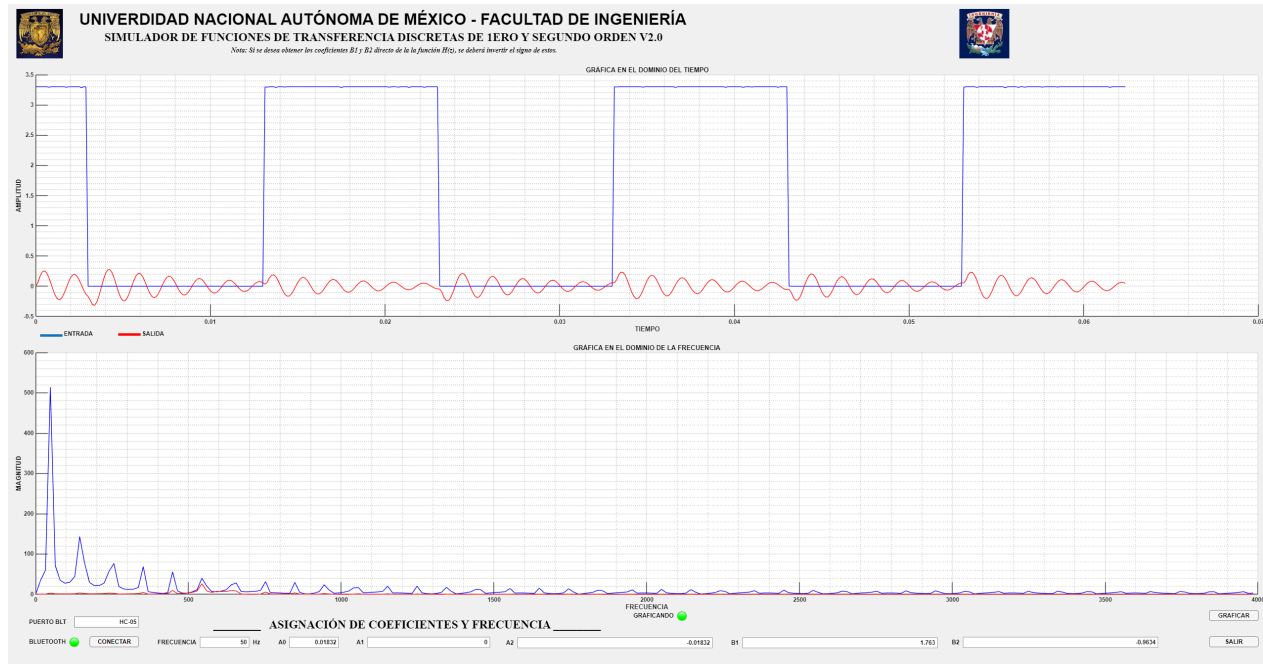


Figura 14: Respuesta del filtro pasa banda a 50 Hz. Se observa una fuerte atenuación, ya que la frecuencia de entrada está muy alejada de la banda de paso.

3.3.2. Prueba con $f = 100$ Hz ($f \approx f_o$)

Al incrementar la frecuencia a 100 Hz, aunque todavía nos encontramos lejos de la frecuencia central de diseño, el filtro comienza a mostrar una respuesta ligeramente menos atenuada que en la prueba anterior. A pesar de estar en la zona de transición, la salida $Y(k)$ nos muestra la selectividad del sistema, permitiendo el paso de una mayor amplitud de la señal de entrada a medida que nos acercamos a la banda de paso.

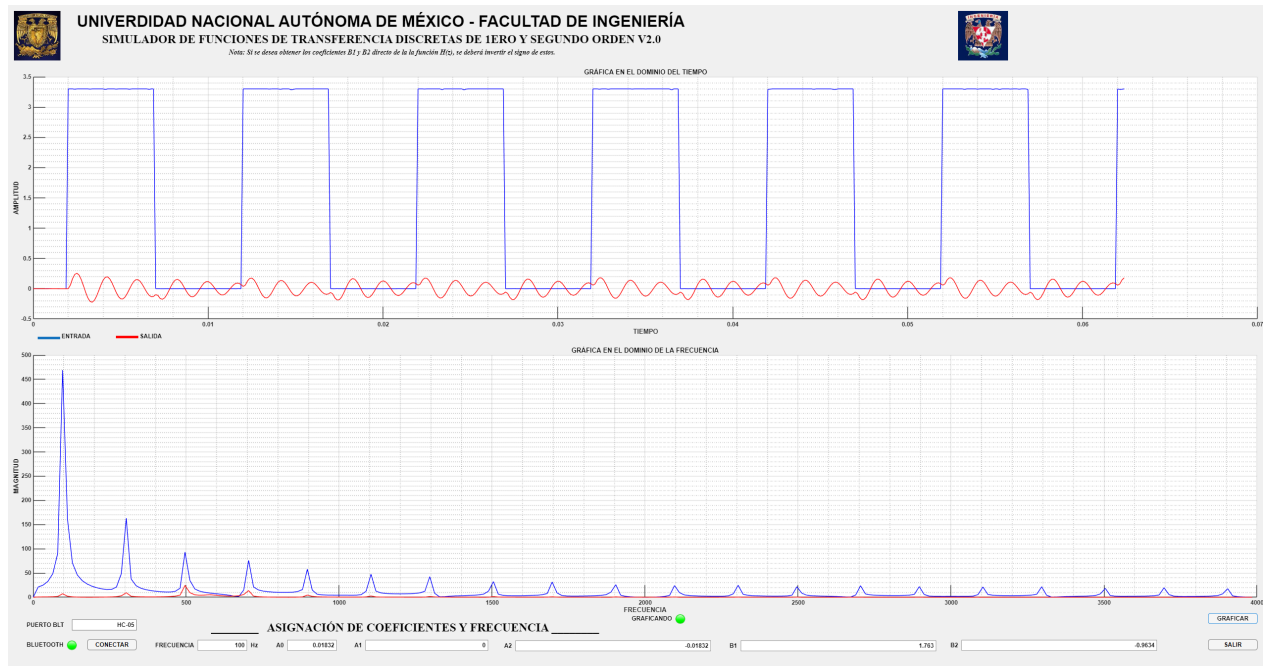


Figura 15: Respuesta del filtro pasa banda a 100 Hz. Se observa una atenuación menor en comparación a la prueba de 50 Hz al acercarse progresivamente a la frecuencia central.



4. Conclusiones:

■ Espinoza Matamoros Percival Ulises:

Al implementar el simulador de funciones de transferencia discretas específicamente para filtros IIR Butterworth en la Raspberry Pi Pico 2W, se logró retomar los conocimientos obtenidos en la práctica anterior, específicamente sobre el proceso de discretización mediante la transformación bilineal, en esta práctica se retomaron conceptos de señales y sistemas así como de sistemas de comunicaciones para poder diseñar los filtros adecuadamente y que estos funcionaran correctamente. El sistema implementado demuestra la capacidad de la Raspberry Pi Pico 2W para realizar procesamiento digital de señales en tiempo real, manteniendo una frecuencia de muestreo constante de 8[kHz], donde para esto con un núcleo se genera la señal cuadrada de prueba y con el otro núcleo se realiza el filtrado y la gestión de datos, posteriormente los datos son enviados a MATLAB a través del módulo Bluetooth HC-05, ofreciendo así una mayor flexibilidad para el análisis y visualización de los resultados, como gráficas en el dominio de la frecuencia y del tiempo, lo que permitió validar la precisión de los filtros implementados.

Es por esto que gracias a la realización de esta práctica logró aprender la importancia y funcionalidad de los filtros digitales IIR, desde su implementación en un microcontrolador como la raspberry pi pico 2w por medio de discretizar la función de transferencia de cada filtro, hasta la importancia de mantener una frecuencia de muestreo estable para garantizar un filtrado efectivo, la capacidad de procesamiento que ofrece la Raspberry Pi Pico 2W al tener dos núcleos para distribuir tareas, así como la importancia de los protocolos de comunicación para garantizar la integridad de los datos al transmitirlos a través de Bluetooth, permitiendo así diseñar e implementar sistemas más complejos que requieran una óptima recuperación y análisis de señales en tiempo real.

■ Flores Colin Victor Jaziel:

El objetivo de diseñar e implementar un simulador de funciones de transferencia discretas se cumplió satisfactoriamente ya que logramos con éxito el procesamiento digital de señales con la implementación en la Raspberry Pi Pico 2W, apoyada por MicroPython, demostró que es posible realizar filtrado en tiempo real manteniendo la rigurosidad matemática requerida para la transformación de modelos continuos a discretos. El diseño



del sistema tuvo un enfoque sistémico para garantizar que la frecuencia de muestreo de 8 kHz se mantuviera constante ante la carga computacional del filtrado y la gestión del tráfico de datos. La integración entre MATLAB y el hardware a través del protocolo Bluetooth HC-05 fue importante validar el desempeño del simulador y al modificar los parámetros, este enlace inalámbrico transformó una implementación estática en una herramienta de análisis versátil, para contrastar de forma inmediata el comportamiento en el dominio del tiempo con el análisis espectral en la interfaz gráfica. La visualización clara de la respuesta de cada tipo de filtro pasa bajas, pasa altas y pasa banda, ante una señal de entrada, confirmó la precisión de los parámetros calculados mediante la transformación bilineal. Esta práctica me permitió consolidar la teoría de sistemas discretos dentro de una arquitectura real. El proyecto no solo validó la eficacia de los filtros Butterworth implementados, sino que también se destacó la importancia de optimizar la ejecución multinúcleo y la sincronización de datos para evitar la latencia y la pérdida de información, cumpliendo así con los estándares de diseño propuestos para el control y análisis de señales en tiempo real.

■ **García Cortés Adolfo de Jesús:**

En conclusión, gracias al desarrollo de la práctica se logró implementar filtros digitales IIR Butterworth de manera funcional utilizando la Raspberry Pi Pico 2W. Al probar los códigos, se logró comprobar que el comportamiento real de los filtros coincidió con lo esperado teóricamente, obteniendo respuestas correctas para los filtros pasa bajas, pasa altas y pasa banda.

La transformación bilineal y la técnica de *pre-warping* ayudaron a mantener correctamente las frecuencias de corte definidas en el diseño del filtro. Gracias a esto, el filtro pasa bajas suavizó la señal, el pasa altas detectó cambios rápidos y el pasa banda dejó pasar únicamente un rango específico de frecuencias.

El uso de los dos núcleos de la Raspberry Pi Pico 2W permitió separar tareas, de manera que un núcleo generaba la señal de prueba mientras el otro realizaba el filtrado y el envío de datos, evitando problemas de ejecución.

Además, se logró mantener una frecuencia de muestreo estable de 8 kHz usando un control preciso del tiempo con `ticks_us`, lo que permitió que el filtro trabajara de manera continua y uniforme.



Por otro lado, la comunicación con el módulo Bluetooth HC-05 funcionó correctamente gracias al envío de datos en pequeños bloques, evitando pérdidas de información durante la transmisión.

En general, esta práctica permitió aplicar conocimientos de procesamiento digital de señales, programación en MicroPython, comunicación serial y programación multinúcleo, demostrando cómo los filtros digitales pueden implementarse de manera práctica y eficiente.

■ **Lara Hernandez Angel Husiel:**

El diseño de los filtros mediante la transformada bilineal con pre-distorsión de frecuencias demostró ser un método sólido para trasladar el comportamiento analógico al dominio discreto sin que los coeficientes perdieran precisión. Calcular cada término a mano fue lo que permitió entender de dónde viene realmente cada valor que se carga al microcontrolador, algo que difícilmente se aprecia cuando todo lo resuelve una función directa de MATLAB. La implementación en la Raspberry Pi Pico 2W añadió una capa adicional de complejidad al tener que garantizar un periodo de muestreo estable de $125\ \mu\text{s}$, distribuir tareas entre los dos núcleos del procesador y diseñar un protocolo de comunicación Bluetooth que no saturara el canal al momento de transmitir los datos capturados.

La comunicación entre el módulo HC-05 y MATLAB operando a 460,800 baudios resultó ser un factor determinante en la estabilidad del sistema. Esa velocidad de transmisión tuvo que equilibrarse con las pausas de 60 ms entre bloques de datos para evitar que el buffer del módulo se desbordara, lo que de no haberse considerado habría provocado pérdida de muestras y resultados incoherentes en las gráficas. El hecho de que el canal inalámbrico pudiera mantenerse estable durante las capturas, actualizando coeficientes y frecuencia en tiempo real sin interrumpir el procesamiento, confirmó que el diseño del protocolo de comunicación fue adecuado para las exigencias del sistema.

Es por esto, que ya en matlab, tenemos el primer filtro, que en el pasa bajas produjo una salida que sigue lentamente a la onda cuadrada con curvas exponenciales cuya velocidad depende directamente de qué tan cerca está la frecuencia de prueba de la frecuencia de corte, mientras que el pasa altas reaccionó únicamente a las transiciones, generando picos que decaen hacia cero entre un flanco y otro. En el dominio de la frecuencia esto se



confirmó con espectros que mostraron con claridad qué armónicos de la señal cuadrada sobrevivían al filtro y cuáles quedaban eliminados. La correspondencia entre lo que se veía en el tiempo y lo que aparecía en el espectro fue la verificación más directa de que el sistema funcionó correctamente de extremo a extremo.

■ **Lugo Manzano Rodrigo:**

Esta practica me permitió pasar varios conceptos teóricos, incluso algunos aprendidos en asignaturas anteriores, aplicandolos en la práctica como el procesamiento digital de señales hacia una implementación funcional en hardware. Diseñar estos filtros Butterworth de primer y segundo orden mediante la transformación bilineal fue el primer paso, pero también uno de los retos estuvo en resolver los desafíos técnicos en la ejecución en tiempo real. Para asegurar un periodo de muestreo constante de 125 microsegundos a una frecuencia de 8 kHz, fue importante aplicar una temporización en MicroPython utilizando funciones como `ticks_us` y `ticks_diff`, evitando el desfase temporal acumulativo que sucede con retardos fijos. Asimismo, la optimización mediante el decorador `@micropython.native` e inicializar referencias locales dentro de la función de captura nos permitio incrementar la velocidad de procesamiento de la ecuación en diferencias en el núcleo principal.

La distribución de tareas en esta arquitectura multinúcleo resultó útil para la estabilidad general del simulador, asignar de forma independiente la generación de la onda cuadrada al segundo núcleo mediante la biblioteca `thread` garantizó que la conmutación digital de los flancos no interfiriera con la carga computacional del filtrado. Toda esta infraestructura se integró con la etapa de comunicación inalámbrica a través del módulo HC-05, donde la configuración UART a 460800 baudios y la transmisión segmentada en bloques de 50 bytes con pausas de 60 milisegundos previnieron el desbordamiento del buffer al enviar los arreglos de datos hacia MATLAB. Lo realizado en esta práctica me enseñó la importancia de la sincronización de procesos y la optimización de código, demostrando que una gestión del tiempo y la memoria permite diseñar herramientas de análisis espectral eficientes y dinámicas.



Referencias

- Butterworth, S. (1930). On the Theory of Filter Amplifiers. *Wireless Engineer*, 7(6), 536-541.
- MicroPython Contributors. (2024a). *_thread — Multithreading Support*. https://docs.micropython.org/en/latest/library/_thread.html
- MicroPython Contributors. (2024b). *class ADC — Analog to Digital Conversion*. <https://docs.micropython.org/en/latest/library/machine.ADC.html>
- MicroPython Contributors. (2024c). *class PWM — Pulse Width Modulation*. <https://docs.micropython.org/en/latest/library/machine.PWM.html>
- MicroPython Contributors. (2024d). *machine — Functions Related to the Hardware*. <https://docs.micropython.org/en/latest/library/machine.html>
- MicroPython Contributors. (2024e). *time — Time Related Functions*. <https://docs.micropython.org/en/latest/library/time.html>
- Ogata, K. (2010). *Modern Control Engineering* (5.^a ed.). Pearson.
- Python Software Foundation. (2024). *_thread — Low-Level Threading API*. https://docs.python.org/3/library/_thread.html
- Raspberry Pi Ltd. (2024a). *Raspberry Pi Pico 2 W Datasheet*. <https://datasheets.raspberrypi.com/pico/pico-2-w-datasheet.pdf>
- Raspberry Pi Ltd. (2024b). *RP2350 Datasheet: A Microcontroller by Raspberry Pi*. <https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf>
- Sedra, A. S., Smith, K. C., Carusone, T. C., & Gaudet, V. (2020). *Microelectronic Circuits* (8.^a ed.). Oxford University Press.